COMP3122

Final Project Report

STEM Learning Platform with GenAI

Prepared by	Project Team
Date	April 2026
Repo snapshot	2026-04-05

This report documents the current implemented state of the repository, including the web application, Python backend, Supabase schema and Edge Functions, testing strategy, deployment topology, and role-based user workflows.

Contents

1 Executive Summary **3**

2 Project Background and Motivation **3**

2.1 Course and problem context 3

2.2 Project goals 4

2.3 What makes this project distinctive 4

3 System Overview and Implemented Features **5**

3.1 High-level system picture 5

3.2 Primary user groups 5

3.3 Feature set 6

3.4 Blueprint-centered value flow 6

4 System Design **7**

4.1 Technology stack 7

4.2 Architectural decomposition 7

4.3 Web application design 8

4.4 Python backend design 8

4.5 AI orchestration design 9

4.6 Chat workspace and memory design 9

4.7 Material processing pipeline 10

4.8 Analytics and adaptive teaching support 11

4.9 Security and access-control design 11

4.10 Guest sandbox design 12

5 Software Development Process **12**

5.1 Requirements and scope shaping 12

5.2 Design process 13

5.3 Implementation phases observed in git history 13

5.4 Collaboration workflow 14

5.5 Use of GenAI during development 15

6 Testing, Quality Assurance, and Deployment **15**

6.1 Testing strategy 15

6.2 Why the testing approach is appropriate 16

6.3 Deployment topology 17

6.4 Deployment process 17

6.5 Database evolution 18

7 Lessons Learned and Reflection **18**

7.1 Technical lessons 18

7.2 Process lessons 19

7.3 What the project did well 19

7.4 What could still be improved 19

8 User Manual **20**

8.1 Administrator / System Maintainer 20

8.2 Instructor / Grader Manual 21

8.3 Student Manual 23

8.4 Supplementary Guest Demo Flow 25

9 Conclusion **26**

A Repository Evidence Used for This Report **26**

B Current Accuracy Caveat About the "Admin" Role **27**

1 Executive Summary

This project delivers a production-oriented STEM learning platform that integrates Generative AI into classroom workflows rather than treating AI as a detached chatbot. Its main idea is to let instructors upload teaching materials, convert those materials into a structured *Course Blueprint*, and use that blueprint as the source of truth for downstream student learning activities such as grounded chat, quizzes, flashcards, assignment review, and class-level insights.

The implemented system is split into three cooperating runtime surfaces:

- a **Next.js 16** web application for role-aware user experience, route handling, and server actions;
- a **Python FastAPI** backend for AI orchestration, class workflows, analytics, and chat memory management; and
- a **Supabase** project for authentication, PostgreSQL persistence, row-level security, storage, queue-backed material processing, and guest sandbox lifecycle support.

From a product perspective, the platform supports:

- **instructors / graders** who create classes, upload materials, generate and publish blueprints, create assignments, review submissions, and inspect analytics;
- **students** who join classes with join codes, use the published blueprint, complete assignments, and learn through grounded AI chat; and
- **guest users** who can explore the product through a realistic sandboxed demo flow backed by Supabase Anonymous Auth and sandbox-cloned data.

The repository also demonstrates a meaningful software engineering process. The current snapshot contains 401 commits, 92 merge commits, 23 active database migrations, a multi-layer test strategy, and a documented deployment topology across web, backend, and Supabase services. The result is not a toy prototype, but a coordinated educational platform with clear subsystem boundaries and operational considerations.

2 Project Background and Motivation

2.1 Course and problem context

The supplied project overview and background materials position the project in the context of *Education 4.0*, AI-empowered personalized learning, and the practical pain points faced by teachers. The brief explicitly asks teams to avoid building "just another AI Q&A app" and instead create a platform with meaningful educational value, differentiated features, and a well-considered software engineering process.

The background deck highlights several recurring teaching needs:

- reducing instructor workload in lesson preparation and grading;
- generating learning activities such as quizzes and guided exercises;
- using AI to support more personalized and interactive learning;
- grounding AI output in trustworthy course content rather than generic internet-style chat.

These needs strongly influenced the design of this system. Rather than exposing a general-purpose chatbot, the platform is built around a teacher-controlled workflow where uploaded course materials are curated into a blueprint before AI-generated learning activities are released to students.

2.2 Project goals

Based on the repository documentation and implementation, the core goals of the project are:

- transform raw teaching materials into structured, reusable course knowledge;
- keep instructors in editorial control of AI outputs;
- provide students with guided, class-specific learning experiences;
- support realistic classroom operations such as assignment creation, review, and feedback;
- demonstrate a robust engineering process including testing, deployment, and iteration.

2.3 What makes this project distinctive

Several aspects make this project more substantial than a basic GenAI demo:

- the **Course Blueprint lifecycle** introduces an approval and publishing checkpoint;
- **all AI generation routes through a dedicated Python service**, not ad hoc frontend calls;
- **material processing is asynchronous and queue-backed**, which is closer to production reality;
- **teacher intelligence features** extend beyond content generation into analytics and adaptive briefs;
- **guest mode** is implemented as a true sandboxed experience rather than a fake slideshow demo.

3 System Overview and Implemented Features

3.1 High-level system picture

Figure 1 shows the top-level structure of the current platform.

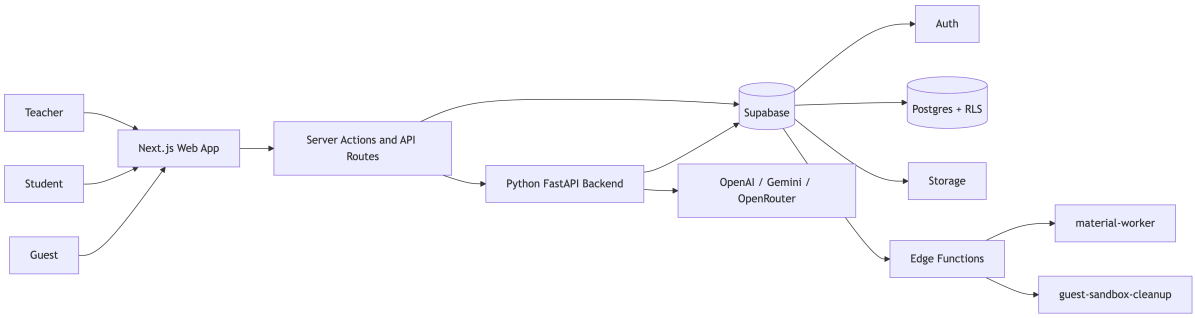

Figure 1: Current high-level platform architecture.

At a high level, teacher, student, and guest users all enter through the web application. The web layer handles UI rendering and user flow orchestration, while the backend is responsible for AI-heavy logic and the Supabase layer provides persistence, storage, security, and async jobs.

3.2 Primary user groups

Table 1: Primary user groups in the current release

Role	Implemented responsibilities
Instructor / Grader	Create classes, upload materials, generate and curate blueprints, create assignments, review student work, monitor chat activity, inspect class insights, use the adaptive teaching brief, and preview the student experience.
Student	Join classes using a join code, access the published blueprint, use always-on AI chat, complete chat/quiz/flashcards assignments, and track progress from the student dashboard.
Guest	Enter a sandbox demo, explore both teacher and student views through role switching, and safely evaluate the product without touching real user data.

3.3 Feature set

Table 2: Major implemented product features

Feature area	Current implementation
Authentication	Email/password sign-in for permanent accounts, email verification, password reset, and guest access via Supabase Anonymous Auth.
Class management	Teacher class creation, student join-by-code, role-aware dashboards, and class-scoped navigation.
Materials workflow	PDF, DOCX, and PPTX upload; asynchronous extraction/chunking/embedding; preview, download, and delete actions.
Blueprint workflow	Draft, overview, and published blueprint lifecycle with teacher editing and publish control.
Learning activities	Chat assignments, quiz assignments, flashcard assignments, plus open-practice class chat.
Teacher review	Assignment review screens for chat, quiz, and flashcards, including feedback-saving flows.
Analytics	Class intelligence dashboard with summaries, charts, student drill-downs, and intervention suggestions.
Teaching support	Adaptive teaching brief widget summarizing what needs attention and recommended next actions.
Visual explanation	Generative canvas support for charts and diagram-style responses in chat and analytics contexts.
Guest mode	Sandbox provisioning, role switching, reset flow, quota management, expiry, and cleanup.

3.4 Blueprint-centered value flow

The most important end-to-end educational flow is shown in Figure 2.

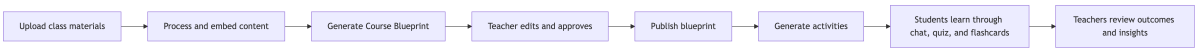

Figure 2: Blueprint-centered value flow from teacher materials to student learning and review.

This flow is central to the platform's pedagogy:

1. the instructor uploads materials;
2. the system processes and embeds those materials;
3. the AI generates a structured blueprint;
4. the instructor edits and approves the blueprint;

5. the published blueprint becomes the basis for activities;
6. students learn through grounded interactions;
7. instructors review outputs and analyze class performance.

4 System Design

4.1 Technology stack

Table 3: Core implementation stack

Layer	Technology
Frontend	Next.js 16 App Router, React 19, TypeScript, Tailwind CSS 4, shared UI primitives, Motion for React, Lucide icons
Backend	Python FastAPI, httpx, structured Pydantic schemas, provider adapters
Database and auth	Supabase PostgreSQL, Supabase Auth, Row Level Security
Storage and async processing	Supabase Storage, pgmq queue, pg_cron, Supabase Edge Functions
Testing	Vitest, React Testing Library, Python unittest, Playwright
Deployment model	Separate web and backend deployments plus hosted Supabase

4.2 Architectural decomposition

The current repository is intentionally partitioned into four main top-level areas:

- `web/`: user-facing routes, UI components, middleware, and server actions;
- `backend/`: AI orchestration, chat workspace logic, class workflows, analytics;
- `supabase/`: SQL migrations, local config, and Edge Functions;
- `tests/`: Playwright end-to-end coverage running against a deployed target URL.

This separation matters because different responsibilities are intentionally placed in the layer best suited for them. The web app owns rendering and navigation; the Python backend owns AI and workflow-heavy service logic; Supabase owns security, persistence, storage, and background jobs.

4.3 Web application design

The web application is not just a collection of pages. It implements a role-aware interface model with separate teacher and student dashboards, a persistent class shell, and route-level auth protection.

Important web-layer characteristics include:

- role-specific dashboards at `/teacher/dashboard` and `/student/dashboard`;
- unified auth via a shared `AuthSurface` used in modal and dedicated-page forms;
- middleware-enforced redirects for unauthenticated, unverified, and expired guest sessions;
- server actions for class creation, join-by-code, materials, assignments, and settings;
- a shared design system with centralized icons, tokens, and motion presets.

On the teacher side, the class overview acts as a control hub for materials, assignments, teaching brief, and monitoring. On the student side, the class workspace is organized around focus widgets for AI chat, assignments, and the published blueprint. A teacher can also preview the student experience without changing accounts.

4.4 Python backend design

The backend is the sole AI orchestration boundary in the current implementation. This is an important design choice because it avoids scattering prompt construction, provider credentials, and response normalization across the frontend.

The backend currently provides endpoints and domain logic for:

- blueprint generation;
- quiz generation;
- flashcard generation;
- grounded chat;
- chat workspace sessions and message persistence;
- visual canvas generation;
- class create and class join;
- materials dispatch;
- analytics, teaching brief, and natural-language data queries.

All normal backend responses follow a consistent envelope:

```
{ ok, data, error, meta }
```

This design improves consistency at the integration boundary. The frontend adapters can handle success and failure cases uniformly, while the backend retains freedom to evolve internal provider logic and prompt design.

4.5 AI orchestration design

The repository currently supports OpenRouter, OpenAI, and Gemini as pluggable providers. Provider choice is environment-driven, and the backend can fall back across providers when needed.

This design has several advantages:

- the web app remains provider-agnostic;
- deployment environments can switch provider preference through configuration;
- provider failures can be isolated inside the backend;
- request deadlines are enforced centrally rather than inconsistently per route.

The current AI domains are broader than content generation alone. In addition to blueprint, quiz, and flashcard generation, the system also supports analytics narratives, teaching brief synthesis, chat memory handling, and visual canvas specifications.

4.6 Chat workspace and memory design

One of the most technically interesting parts of the system is the chat workspace. It is not implemented as a purely stateless prompt-response loop. Instead, the backend includes a dedicated chat workspace subsystem with participant listing, session creation and archive, paginated message history, and context compaction for long-running conversations.

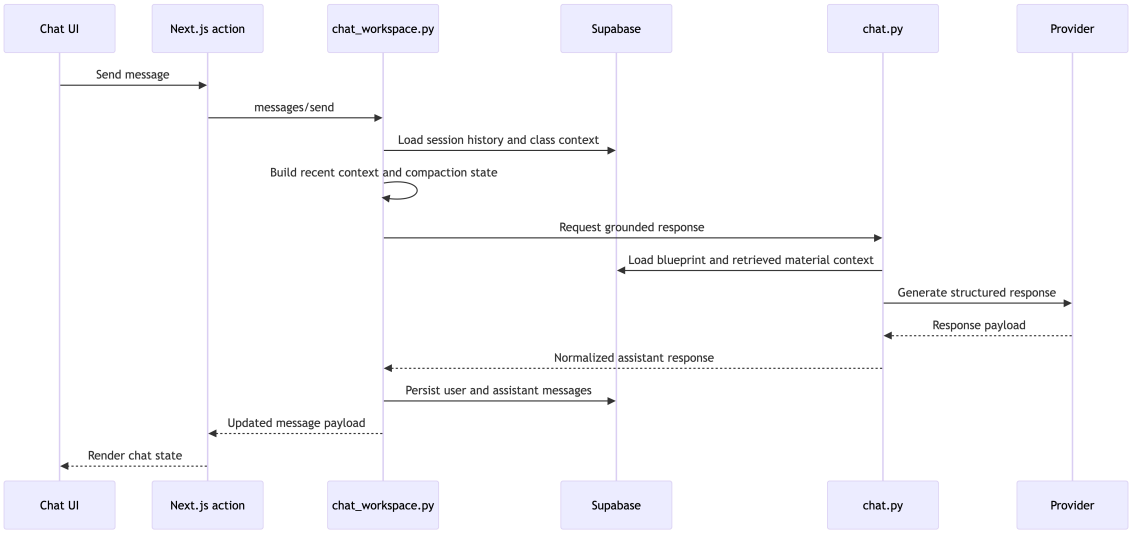

Figure 3: Chat workspace flow with grounded context retrieval and persisted assistant responses.

This design is significant for two reasons. First, it better reflects real classroom use, where students may return to a conversation over time rather than ask only one question. Second, it forces the system to deal with long-context management, not just one-shot prompt construction.

4.7 Material processing pipeline

Material ingestion is intentionally asynchronous. Uploading a file creates a material record and stores the file, but extraction, chunking, and embedding happen through a queue-backed worker path instead of blocking the user's request.

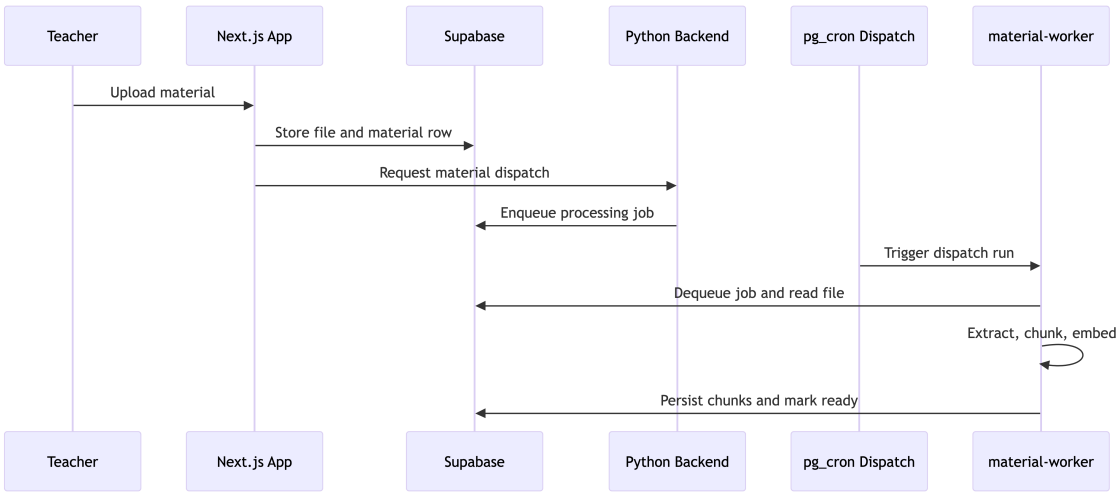

Figure 4: Asynchronous material-processing pipeline.

This was a good architectural decision for a course project because it addresses a real system concern: extraction and embedding are too slow and failure-prone to treat as

a trivial synchronous request. The queue-backed design allows better retry behavior, clearer UI states (`processing`, `ready`, `failed`), and a cleaner separation between user interaction and background work.

4.8 Analytics and adaptive teaching support

The platform includes teacher-facing intelligence features, which meaningfully extend it beyond content generation.

The class intelligence dashboard includes:

- summary cards for student count, average score, completion rate, and risk indicators;
- topic-level performance charts;
- Bloom-level breakdown visualization;
- student engagement views and student overview tables;
- AI-generated narrative interpretation and intervention suggestions;
- natural-language data query support for generating chart specifications.

The adaptive teaching brief is a complementary feature. Rather than exposing a large dashboard all the time, it provides a concise memo-style summary directly on the class overview page, with recommended next steps, likely misconceptions, and follow-up activity suggestions.

4.9 Security and access-control design

Security is handled across all three runtime surfaces:

- **web layer**: route protection, email verification enforcement, guest route confinement;
- **backend layer**: service authentication, user bearer-token verification, guest AI guardrails;
- **Supabase layer**: row-level security, teacher/enrollment policies, and sandbox-aware isolation.

The current implementation documents several security-conscious choices:

- permanent users are limited to teacher or student account types;
- `profiles.account_type` is immutable after signup;
- guest sessions are scoped by `sandbox_id`;
- cursor inputs to PostgREST are validated before interpolation;
- service-to-service auth between web and backend uses a shared backend API key.

4.10 Guest sandbox design

Guest mode is one of the strongest differentiators in the project. Instead of presenting a static marketing demo, the system provisions a real anonymous session and clones a seeded classroom into a sandboxed data space. Guests can switch between teacher and student views, reset the demo, and observe realistic product behavior without accessing real user data.

The current quota model enforces:

- a global cap on active guest sessions;
- a per-hour cap on newly created guest sessions;
- backend limits on guest AI usage and concurrent in-flight AI requests;
- expiry after 32 hours maximum or 8 hours of inactivity.

This is a thoughtful design because it balances demo accessibility with safety, resource control, and cleanup reliability.

5 Software Development Process

5.1 Requirements and scope shaping

The initial project brief encouraged teams to investigate stakeholder needs, review existing platforms, and avoid shallow AI-first implementations. The current repository reflects that direction. The system is not scoped as a universal learning platform, but as a structured STEM classroom assistant centered on teacher materials, editorial control, and student-facing learning activities.

A practical strength of the project is its restraint. The team did not attempt to implement every possible educational feature. Instead, it converged on a coherent workflow:

- content ingestion;
- blueprint generation and publishing;
- student chat and assignments;
- review and analytics;
- guest evaluation flow.

This focused scope likely helped the team build a more integrated product instead of a fragmented set of unrelated AI experiments.

5.2 Design process

The design process visible in the repository appears iterative and documentation-driven. The project contains:

- top-level design, architecture, UI/UX, and deployment documents;
- internal reference notes under `.codex/`;
- Mermaid diagrams exported to PNG for communication and reporting;
- commit history showing multiple phases of refinement rather than one-shot implementation.

The UI/UX work also appears to have been treated as a first-class concern. The repository contains significant evidence of design-token work, navigation restructuring, auth UX redesign, teacher/student flow differentiation, and consistency hardening. This is valuable because educational software quality depends not only on AI correctness, but also on clarity, trust, and ease of use.

5.3 Implementation phases observed in git history

The commit history provides a useful implementation timeline. Table 4 summarizes the major development phases visible from the repository.

Table 4: Major development phases observed from git history

Period	Main progression visible in the repository
2026-02-03 to 2026-02-05	Repository setup, initial schema and RLS, core class flows, materials pipeline, AI adapters, blueprint generation, and first UI scaffolding.
2026-02-06 to 2026-02-10	Vertical slices for AI chat, quiz generation, flashcards, auth hardening, and always-on class chat workspace.
2026-02-22 to 2026-03-02	Material worker stabilization, queue-backed processing, dashboard and class-shell UX improvements, theme/token refactoring, and stronger unit-test coverage.
2026-03-15 to 2026-03-24	Teacher class intelligence dashboard, generative canvas, teaching brief backend and widget, content review improvements, and more polished materials preview behavior.
2026-03-26 to 2026-03-28	Guest mode foundation and hardening, E2E expansion, shared auth redesign, email template work, OTP-based password change flow, and documentation refresh.
2026-04-04 to 2026-04-05	Guest quota redesign, concurrency improvements, final bug fixing, environment sync, and diagram/documentation alignment with runtime reality.

This sequence suggests a healthy development pattern:

1. build the skeleton;
2. deliver core vertical slices;
3. strengthen architecture and UX;
4. add analytics and guest mode;
5. harden the system and documentation near submission.

5.4 Collaboration workflow

The collaboration model that is directly observable from the repository is based on GitHub-style branching, pull requests, review, and merge integration.

Evidence visible in the current snapshot includes:

- 401 commits in total;
- 92 merge commits;
- repeated feature-branch merge patterns such as guest mode, UI improvements, analytics, teaching brief, and auth flows;
- regular documentation updates that track design decisions, rollout notes, and lessons learned.

This indicates an iterative workflow rather than a single large code dump. The team appears to have used:

- feature branches for major workstreams;
- pull requests to integrate features and fixes;
- tests and documentation updates alongside implementation changes;
- structured plans and handoff notes for ongoing work.

At the same time, one accuracy caveat is important: the visible git author history is concentrated under a small number of identities, especially one primary contributor identity. For that reason, this report documents the *repository-observable collaboration process* rather than making unsupported claims about each team member's exact offline contribution split. If the course requires a per-member effort table, that should be appended using the team's own meeting notes or task records.

5.5 Use of GenAI during development

This project is itself a GenAI platform, but the repository also suggests that GenAI-assisted development was part of the engineering workflow. The documentation explicitly discusses Codex- and agent-assisted workflows, planning notes, and iterative bug fixing. From a software engineering perspective, this is useful to reflect on because GenAI appears to have been used as:

- a coding accelerator for scaffolding and incremental implementation;
- a design aid for exploring architecture and UI alternatives;
- a debugging assistant for reproducing and fixing workflow bugs;
- a documentation aid for maintaining architecture and usage guides.

However, the repository also shows good evidence of human judgment:

- repeated refactors and bug-fix passes after initial implementations;
- hardening around access control, redirect behavior, and quota logic;
- correction of real production-style issues such as signed URL previewing, timeout handling, guest cleanup, and environment misconfiguration;
- a sustained effort to align documentation with the actual runtime state.

This combination is exactly the kind of responsible GenAI-assisted development that a modern course project should demonstrate: AI helps with speed, but humans still own design quality, correctness, and final judgment.

6 Testing, Quality Assurance, and Deployment

6.1 Testing strategy

The repository implements a layered testing strategy rather than relying on only one test type.

Table 5: Current testing layers in the repository

Layer	What it covers	Evidence in repo
Web unit and integration tests	Server actions, auth flows, route guards, material actions, blueprint logic, chat flows, guest helpers, UI primitives, validation helpers	60 test files under `web/src`
Backend unit tests	Providers, schemas, class flows, blueprint generation, quiz, flashcards, materials, analytics, chat workspace, guest rate limiting	17 Python test files under `backend/tests`
End-to-end tests	Teacher navigation, student navigation, login, join class, settings, sign-out, and guest entry against a deployed environment	9 Playwright specs under `tests/e2e`

Representative quality signals include:

- guest middleware tests and guest sandbox tests;
- blueprint action tests;
- materials extraction and retrieval tests;
- chat generation, compaction, and workspace tests;
- analytics and teaching brief action tests;
- Playwright flows for both teacher and student navigation.

6.2 Why the testing approach is appropriate

This testing distribution is appropriate for the architecture:

- unit tests catch domain logic regressions close to the code;
- web integration tests validate server actions and route behavior;
- backend tests verify AI-related orchestration and guardrails without needing full browser runs;
- Playwright tests confirm that the deployed user flows still work end to end.

For a project involving multiple services and stateful workflows, this layered approach is far more credible than relying on only manual demonstration.

6.3 Deployment topology

The deployed system consists of a frontend deployment, a separate backend deployment, and a hosted Supabase project. Figure 5 captures this structure.

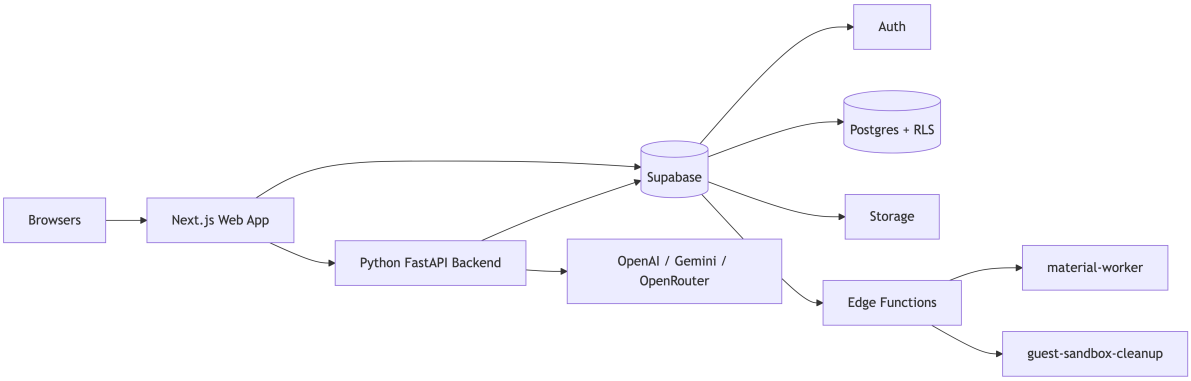

Figure 5: Hosted deployment topology across frontend, backend, and Supabase services.

This split has practical benefits:

- frontend issues can be distinguished from backend issues;
- AI orchestration can evolve independently from the UI deployment;
- Supabase-specific operational tasks such as migrations and Edge Function secrets remain explicit;
- the team can test preview or staging environments in a more controlled way.

6.4 Deployment process

The documented deployment workflow includes:

1. configuring one Supabase project per environment;
2. applying the active migration set;
3. deploying the `material-worker` and `guest-sandbox-cleanup` Edge Functions;
4. configuring Edge Function secrets and Vault-backed values;
5. deploying the Python backend;
6. deploying the web application;
7. running post-deploy smoke tests for auth, teacher flows, student flows, insights, and guest mode.

Operationally, this is a mature approach for a course project because it recognizes that deployment is not just “push frontend to Vercel”; the data plane and async workers also have to be correct.

6.5 Database evolution

The current migration history shows 23 active migrations, which is another sign that the project evolved iteratively rather than remaining static after an initial schema dump.

Major migration themes include:

- initial schema and RLS foundation;
- quiz integrity and auth hardening;
- canonical blueprint snapshots;
- always-on class chat and compaction;
- queue-backed material processing;
- class insights and teaching brief snapshots;
- guest mode schema, seed data, cleanup, and quota redesign.

This migration history is important because it mirrors the product's evolution in a traceable, reviewable form.

7 Lessons Learned and Reflection

7.1 Technical lessons

The repository's maintained documentation highlights several concrete lessons that are worth reflecting on.

- **A clean architecture boundary matters.** Routing all AI through the Python backend keeps provider-specific complexity out of the UI and gives the system a stable application-facing contract.
- **Async processing is necessary for realistic content pipelines.** Material extraction and embedding do not belong in a naive synchronous upload request if the goal is a reliable system.
- **Guest mode is harder than it looks.** A believable demo mode requires session quotas, expiry, cleanup, sandbox isolation, and backend-side ownership checks. A demo that looks simple in the UI can still be architecturally complex.
- **Deployment secrets must live in the right place.** The project explicitly notes that Supabase Edge Functions cannot rely on Vercel secrets or Supabase Vault alone; function-level secrets must be configured in Supabase Edge Function Secrets.
- **Subtle browser and CSS issues can become product bugs.** Examples in the repository include iframe preview limitations caused by storage response headers and dialog-centering issues caused by Tailwind v4's use of the standalone CSS `translate` property.

7.2 Process lessons

From a software engineering process perspective, several lessons stand out:

- keeping design, architecture, deployment, and UI docs current reduces confusion late in the project;
- documentation is especially valuable when a project spans frontend, backend, and database layers;
- a branch-and-merge workflow helps isolate major features such as guest mode and auth redesign;
- testing should expand together with complexity, not after the system is already unstable.

7.3 What the project did well

In its current form, the project demonstrates several strengths:

- a coherent product concept instead of a scattered list of AI features;
- strong teacher-centered workflow design;
- real effort on UI quality and navigation clarity;
- a credible multi-service architecture;
- meaningful testing and deployment documentation.

7.4 What could still be improved

A professional reflection should also note realistic limitations:

- the production UI currently has teacher/student roles but no dedicated admin console;
- the operational surface is sophisticated, so monitoring and observability could still grow further;
- the visible git author concentration suggests that a clearer contribution matrix may still be needed for academic assessment;
- broader activity types beyond chat, quiz, and flashcards remain future-facing rather than fully shipped.

8 User Manual

8.1 Administrator / System Maintainer

Important accuracy note: the current release does *not* expose a dedicated in-app administrator dashboard. The repository does contain administrative capabilities at the database and deployment level (for example, `admin_users` in SQL policies and Supabase admin APIs), but operational administration is performed through deployment tooling, Supabase configuration, and project infrastructure rather than through end-user screens. For that reason, this subsection documents the administrator workflow that actually exists in the current system.

A. Initial platform setup

1. Create and configure the target Supabase project.
2. Enable email/password auth and email confirmation.
3. Enable Anonymous Auth if guest mode is required.
4. Set the Site URL and redirect URLs for local and hosted environments.
5. Apply the active database migrations in `supabase/migrations`.
6. Deploy the two required Edge Functions:
   - `material-worker`
   - `guest-sandbox-cleanup`
7. Configure required secrets for the backend and both Edge Functions.
8. Deploy the web app and Python backend.

B. Key administrator responsibilities

- Maintain environment variables for the web app, backend, and Edge Functions.
- Apply the branded confirmation and recovery email templates in Supabase.
- Ensure `PYTHON_BACKEND_URL` and backend API-key alignment across services.
- Monitor guest-mode capacity and cleanup health.
- Run post-deploy smoke tests after releases.
- Investigate common failure modes such as:
  - guest entry failures;
  - materials stuck in `processing`;
  - auth links redirecting to the wrong domain;
  - web UI loading while AI features fail due to backend or provider issues.

C. Common admin commands

From the current repository documentation:

- Install dependencies:

```
pnpm install
pip install -r backend/requirements.txt
```

- Run locally:

```
pnpm dev
uvicorn app.main:app -app-dir backend -host 0.0.0.0 -port 8001
-reload
```

- Apply migrations:

```
npx supabase db push
```

8.2 Instructor / Grader Manual

A. Create an account and sign in

1. Open the landing page.
2. Choose **Create an account**.
3. Select the **teacher** account type.
4. Enter email and password.
5. Verify the email using the confirmation link.
6. Sign in and open the **Teacher Dashboard**.

B. Create and manage a class

1. From the Teacher Dashboard, click **Create class**.
2. Fill in class title and optional metadata such as subject and level.
3. Submit the form.
4. The system creates the class and generates a join code.
5. Share the join code with students so they can enroll from the student side.

C. Upload teaching materials

1. Open the target class.
2. Use the materials upload form to upload PDF, DOCX, or PPTX files.
3. Wait for the system to process the material.
4. Monitor status as `processing`, `ready`, or `failed`.
5. When the material is ready, use the actions menu to preview, download, or delete it.

D. Generate and publish the Course Blueprint

1. Open the class blueprint page.
2. Trigger blueprint generation after materials are ready.
3. Review the generated summary, topics, and objectives.
4. Edit the blueprint draft where needed.
5. Approve the draft and move it into overview.
6. Publish the blueprint when it is ready for student use.

E. Create learning activities

The current release supports three main activity types:

- chat assignments;
- quizzes;
- flashcards.

To create an activity:

1. Ensure a published blueprint exists.
2. Open the relevant activity creation route from the class page.
3. Generate or configure the activity.
4. Review and edit the content where applicable.
5. Publish or assign it to the class.

F. Review submissions

1. Open an assignment's review page.
2. View student recipients, completion states, and submitted content.
3. For chat assignments, inspect transcript and reflection content.
4. For quiz assignments, inspect scores and answers.
5. For flashcards, inspect review statistics and score summaries.
6. Save teacher feedback for the selected submission.

G. Monitor class progress and teaching support

1. Open the class overview to see the adaptive teaching brief.
2. Refresh the brief when more activity data becomes available.
3. Open **Insights** for the class intelligence dashboard.
4. Review charts, student summaries, AI narrative, and intervention suggestions.
5. Use the natural-language data query panel for custom chart-oriented questions.

H. Preview the student experience

1. Open the class page as a teacher.
2. Activate student preview mode using the student-view route state.
3. Inspect how published blueprint, AI chat, and assignment widgets appear to students.
4. Exit preview mode to return to the teacher control surface.

8.3 Student Manual

A. Create an account and sign in

1. Open the landing page.
2. Choose **Create an account**.
3. Select the **student** account type.
4. Enter email and password.
5. Verify the email using the confirmation link.
6. Sign in and open the **Student Dashboard**.

B. Join a class

1. From the Student Dashboard or sidebar, open **Join class**.
2. Enter the join code provided by the instructor.
3. Submit the form.
4. After joining successfully, open the class workspace.

C. Use the student dashboard

The student dashboard organizes assignments into categories such as:

- due now;
- upcoming;
- completed.

Students can use the dashboard to quickly identify overdue or active work and jump directly into the relevant class or assignment.

D. Navigate the class workspace

Within a class, students can access:

- AI chat;
- chat assignments;
- quizzes;
- flashcards;
- the published blueprint.

The workspace is intentionally more focused and less administratively dense than the teacher view.

E. Use AI chat

1. Open the class page and select the AI chat widget or direct chat view.
2. Ask questions related to the class materials and published blueprint.
3. Review the grounded AI response.
4. Continue the conversation as needed.

If chat is not yet available, the likely reason is that the instructor has not published the blueprint yet.

F. Complete assignments

1. Open the assigned activity from the dashboard or class workspace.
2. For chat assignments, complete the conversation and submit the required reflection.
3. For quizzes, answer the questions and submit the attempt.
4. For flashcards, complete the review session and submit progress.
5. Revisit the assignment later to see updated status such as submitted or reviewed.

G. Use the published blueprint

1. Open the blueprint widget or blueprint page inside the class.
2. Review topics and learning objectives.
3. Use the blueprint as a study guide before attempting quizzes or asking AI chat questions.

H. Manage account settings

1. Open **Settings**.
2. Update display name if needed.
3. Use the two-step password change flow with verification code if required.
4. Open **Help** for role-specific FAQ and suggested next steps.

8.4 Supplementary Guest Demo Flow

Although the course report specifically asks for admin, instructor, and student manuals, the guest workflow is worth documenting because it is a significant implemented feature.

1. From the landing page, choose **Continue as guest** when guest mode is enabled.
2. The system provisions an anonymous authenticated session and clones sandbox data.
3. Explore the classroom in teacher view or switch to student view.
4. Reset the sandbox if a fresh walkthrough is needed.
5. Create a permanent account from a clean state if you want to move into normal usage.

9 Conclusion

The current repository demonstrates a serious course project with a clear educational purpose, a layered architecture, meaningful GenAI integration, and a credible software engineering process. Its strongest design choice is the decision to center the product on a teacher-curated Course Blueprint rather than on generic AI chat alone. That decision gives the entire system coherence: materials, activities, student chat, review, and analytics all connect back to the same core object.

Equally important, the project goes beyond feature implementation and addresses the concerns that often separate prototypes from robust systems: asynchronous processing, security boundaries, role-aware access control, test coverage, deployment topology, and realistic demo-mode isolation. For a final project, this makes the platform not only functional, but also defensible from a software engineering perspective.

A Repository Evidence Used for This Report

This report was prepared against the current repository state using the following main sources:

- top-level documents:
  - `README.md`
  - `ARCHITECTURE.md`
  - `DESIGN.md`
  - `UIUX.md`
  - `DEPLOYMENT.md`
- subsystem documentation:
  - `web/README.md`
  - `backend/README.md`
  - `supabase/README.md`
- course materials:
  - `project_overivew.pdf`
  - `project_background.pdf`
- implementation evidence:
  - key web routes and server actions
  - backend service modules
  - Supabase Edge Functions
  - migration history
  - git commit history and merge history
  - test suites and E2E specs

B Current Accuracy Caveat About the "Admin" Role

The course deliverable asks for an admin user manual, but the current repository does not expose a dedicated end-user admin console. Permanent in-app roles are teacher and student, while guest mode is a sandboxed anonymous access path. Administrative responsibilities in the current release exist mainly at the infrastructure and deployment layer. This report therefore documents the administrator workflow that is actually implemented today, rather than inventing an unsupported UI role.